CS 519 Project 3: Part B Report

Saroop Makhija

1 Overview

For Part B, I implemented a simple cooperative scheduling mechanism in the Linux CFS scheduler. The goal was to let an application thread tell the kernel when it is runnable but not doing useful application work. This is different from a blocked thread because the thread is still consuming CPU time. It is also different from `sched_yield()`, because `sched_yield()` only gives up the CPU for one scheduling decision, while my mechanism stores persistent scheduler-visible state until the thread clears it.

The design adds one new system call: `coop_sched(int inactive)`.

The meaning is:

- `coop_sched(1)` marks the calling thread as cooperatively inactive.
- `coop_sched(0)` marks the calling thread as active again.

Invalid values return `-EINVAL`. The cooperative state is stored per thread, not per process.

2 Design

2.1 Why a Per-Thread Flag

I added a new field to `task_struct`: `int coop_inactive`.

This field stores whether the current thread is cooperatively inactive. I made the flag per-thread because Linux represents each pthread with its own `task_struct`. This is important for multithreaded applications. One worker thread may be spinning while another worker thread in the same process is doing real work, so marking the entire process inactive would be too aggressive.

New tasks are initialized as active by setting `p->coop_inactive = 0` in the fork path. This prevents a newly created thread from accidentally inheriting an inactive state from its parent.

2.2 System Call Behavior

The system call is implemented in the scheduler core. It uses `current` to access the calling thread's `task_struct`. It validates the argument, records the thread group ID, updates the inactive flag, and requests rescheduling when a thread marks itself inactive.

Conceptually, the syscall does the following:

- If `inactive` is not 0 or 1, return `-EINVAL`.
- Lock the current task's runqueue.
- Set `current->coop_inactive = inactive`.

- If `inactive == 1`, request rescheduling.
- Unlock the runqueue.
- Return 0.

The runqueue lock is used because the flag affects scheduler decisions. I did not add a new global spinlock. The scheduler already protects runqueue state with the runqueue lock, so using the existing locking structure keeps the design simpler and safer.

The syscall also collects the TGID from `current->tgid`. This lets the kernel identify which application the calling thread belongs to. During testing, I logged the TGID, PID, and inactive value using `pr_info_ratelimited()`.

2.3 CFS Scheduling Policy

Inactive tasks remain in CFS. I did not create a new scheduler class, change nice values, change priorities, change CPU affinity, or modify NUMA behavior.

The scheduling policy is based on preferring active tasks while still keeping inactive tasks runnable. Under normal CFS behavior, the scheduler chooses the runnable entity with the smallest `vruntime`. My modification keeps that behavior as the starting point. If the task chosen by normal CFS is active, the scheduler simply runs it.

If normal CFS chooses an inactive task, the scheduler searches for the active runnable task with the smallest `vruntime`. If such an active task exists, and skipping the inactive task does not exceed the starvation threshold, the scheduler chooses the active task instead. If there is no active task, or if the inactive task has fallen too far behind in `vruntime`, the inactive task is allowed to run.

This means active tasks are preferred over inactive tasks, but inactive tasks are not completely blocked. This was important because if inactive tasks were removed from scheduling entirely, they might never run again and would be unable to reactivate themselves.

To avoid starvation, I used a bounded virtual-runtime gap. The starvation threshold was set to `COOP_STARVE_GRAN_NS = 50000000`, which corresponds to 50 ms. If an inactive task has fallen too far behind in `vruntime`, it is allowed to run. This keeps the cooperative mechanism from permanently starving inactive threads.

2.4 Reactivation Mechanism

Reactivation uses the same syscall. Calling `coop_sched(0)` clears the `coop_inactive` flag and returns the thread to normal CFS behavior.

In the benchmark, each worker thread calls `coop_sched(1)` before entering the spin loop and calls `coop_sched(0)` before exiting. In a real application, a worker thread would call `coop_sched(0)` when real work becomes available.

3 Kernel Changes

The kernel changes were made in the following files:

File	Change
include/linux/sched.h	Added coop_inactive to task_struct.
kernel/fork.c	Initialized new tasks with coop_inactive = 0.
include/linux/syscalls.h	Added the syscall prototype.
arch/x86/entry/syscalls/syscall_64.tbl	Added syscall number 449.
kernel/sched/core.c	Implemented coop_sched.
kernel/sched/fair.c	Modified CFS selection to prefer active tasks.

The syscall number used for testing was 449.

4 Benchmarks

4.1 System Configuration

The experiments were run with the following setup:

- Kernel: 5.15.199
- CPU cores: 16
- Matrix size: 5000 x 5000
- Matrix worker processes: 16
- Spinner threads: 16
- Spinner duration: 60 seconds

The matrix application was the shared-memory matrix multiplication program from Project 1. I used it as the active real-work application. The spinner benchmark was used as the competing inactive workload.

4.2 Spinner Benchmark Modes

The spinner benchmark supports three modes:

- normal: each thread spins normally.
- yield: each thread repeatedly calls sched_yield() while spinning.
- coop: each thread calls coop_sched(1) before spinning and coop_sched(0) before exiting.

The matrix program itself does not call coop_sched(), because it represents the useful workload that should be protected. The cooperative signal is only used by the spinning benchmark.

5 Correctness Checks

I first tested the new syscall directly. The test program confirmed that:

- `coop_sched(1)` succeeded.
- `coop_sched(0)` succeeded.
- `coop_sched(2)` failed with `EINVAL`.

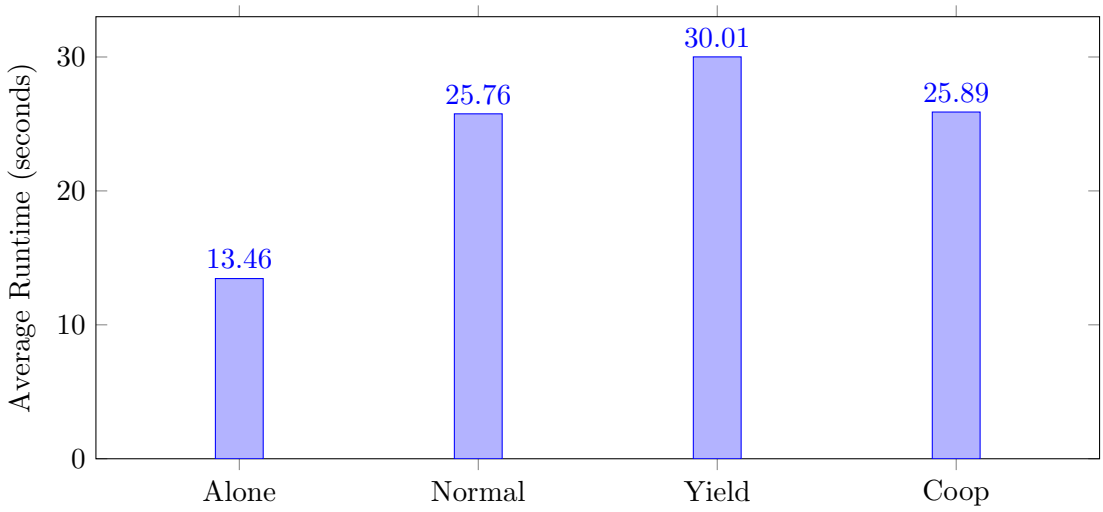
The kernel log also showed multiple worker thread PIDs under the same TGID calling `coop_sched(1)` and later `coop_sched(0)`. This confirmed that the multithreaded benchmark exercised both the inactive and reactivation paths. The matrix application returned `Verification: PASS` in every run.

6 Experimental Results

The table below shows the matrix multiplication runtime under each condition.

Test Mode	Run 1	Run 2	Run 3	Average
Matrix alone	13.408411	13.460558	13.497122	13.455364
Matrix + normal spinner	25.750452	25.712687	25.803064	25.755401
Matrix + `sched_yield()` spinner	29.832023	30.304838	29.884777	30.007213
Matrix + cooperative spinner	25.441771	26.205327	26.021805	25.889634

6.1 Average Runtime Graph



6.2 Slowdown Compared to Matrix Alone

Using the matrix-alone average of 13.455364 seconds as the baseline:

Mode	Slowdown vs. Matrix Alone
Normal spinner	91.41%
`sched_yield()` spinner	123.00%
Cooperative spinner	92.41%

7 Discussion

The cooperative scheduler worked functionally. The syscall correctly updated per-thread scheduler state, invalid arguments were rejected, and the kernel log showed the TGID and PID of calling threads. The benchmark also confirmed that worker threads could mark themselves inactive and later reactivate themselves.

The performance results were mixed. The standalone matrix benchmark averaged about 13.46 seconds. When run with a normal spinning benchmark, runtime increased to about 25.76 seconds. With the cooperative spinner, runtime was about 25.89 seconds, which is very close to the normal spinner case. This means my cooperative policy did not significantly improve matrix performance compared to default behavior.

However, the cooperative spinner performed better than the `sched_yield()` spinner, which averaged about 30.01 seconds. This suggests that repeatedly calling `sched_yield()` inside a spin loop is not a good persistent idleness mechanism. It can create additional scheduling overhead and does not communicate long-term application idleness to the scheduler.

One reason my cooperative scheduler did not produce a large improvement is that the policy was conservative. Inactive tasks remained runnable inside CFS, and the starvation threshold allowed them to keep receiving CPU time if they fell too far behind in virtual runtime. This preserved correctness and avoided starvation, but it also limited how strongly inactive tasks were deprioritized.

Another limitation is that the change only affects CFS task selection on a local runqueue. It does not modify load balancing, CPU affinity, NUMA placement, task weights, or nice values. This makes the implementation small and easier to reason about, but it also means inactive tasks still contribute to runnable load in other parts of the scheduler.

8 Submitted User-Space Tests

I included the following user-space test files:

- `coop_syscall_test.c`: directly tests `coop_sched(1)`, `coop_sched(0)`, and invalid input.
- `spin_bench.c`: multithreaded spinning benchmark with `normal`, `yield`, and `coop` modes.
- `matrix_shm.c`: shared-memory matrix multiplication application from Project 1.
- `run_coop_tests.sh`: test harness for running the benchmark combinations.

The test programs compile with:

```
gcc -O2 matrix_shm.c -o matrix_shm
gcc -O2 -pthread spin_bench.c -o spin_bench
gcc -O2 coop_syscall_test.c -o coop_syscall_test
```

9 Conclusion

In this project, I added a cooperative idleness mechanism to CFS. The design lets each thread mark itself inactive with `coop_sched(1)` and active again with `coop_sched(0)`. The scheduler then prefers active runnable tasks while still allowing inactive tasks to run when necessary to prevent starvation.

The implementation satisfies the assignment requirement without changing nice values. It works for multithreaded benchmarks because the inactive flag is stored per thread. The reactivation path is explicit and was exercised in the benchmark.

The experimental results showed that the implementation works correctly, but the performance benefit was limited. The cooperative spinner behaved similarly to the normal spinner, while `sched_yield()` performed worse. This suggests that the cooperative signal was successfully represented in the kernel, but a more aggressive policy would be needed to provide stronger performance isolation for active applications.